SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Nikhil Kumar Reddy Kadiveti	**Reg. No.:**	192425031
Section / Batch:	2024	**Date:**	

Code of Conduct

I Nikhil Kumar Reddy Kadiveti (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Part of Speech Tagging	Morphological decomposition of analyzing, analysis, and analytical; identification of roots, affixes, inflectional and derivational forms, and normalization for indexing.	Q1
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q2
Counting Words, English Word Classes, Part of Speech Tagging	Morphology-based normalization of govern, government, and governance for topic modeling, clustering, and lexical standardization.	Q3
Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Morphological analysis of activate, activation, and reactivation; identification of derivational layers, word-class changes, and semantic shifts.	Q4
Rule-Based Part of Speech Tagging, English Word Classes, Part of Speech Tagging	Inflectional normalization of create, creates, and creating; extraction of root forms and classification of tense/aspect transformations.	Q5

Annexure A – SCENARIO BASED

1. A large-scale search engine indexing system processes user queries containing word variants such as “analyzing, “analysis”, and “analytical”. The system must improve retrieval accuracy by grouping semantically related terms under a unified representation. Design a morphological processing approach to decompose each input word into root and affixes, distinguish between inflectional and derivational forms, and normalize them for indexing. Apply the process to the given inputs and determine the root form along with the morphological structure of each word.

- Develop a Python program to implement a rule-based morphological processing system for the input words "analyzing", "analysis", and "analytical". The program should identify the root word and separate the corresponding prefixes or suffixes using suitable morphological rules, classify each word as an inflectional or derivational form, normalize all related variants to a common representation for indexing, and generate a structured report showing the original word, extracted root, identified affix(es), transformation type, and normalized output to support efficient search engine retrieval.

2. An AI-based sentiment analysis platform receives product reviews containing words like “disagree”, “agreement”, and “agreeable”. The system must interpret sentiment consistently despite variations in word formation. Develop a morphological parsing strategy to identify prefixes, suffixes, and base forms, while capturing how derivational changes influence meaning. Process the given inputs to extract roots and classify each transformation, ensuring accurate semantic interpretation across all word forms.

- Develop a Python program to create a morphological parser for the input words "disagree", "agreement", and "agreeable". The program should detect and separate prefixes, suffixes, and the base word using rule-based morphological analysis, examine how derivational modifications alter the meaning of the root word, classify each transformation according to its morphological type, and produce a comprehensive output containing the original word, identified prefix, root word, suffix, transformation category, semantic interpretation, and normalized base form for use in sentiment analysis applications.

3. A text analytics pipeline in a news aggregation system handles documents containing terms such as “govern”, “government”, and “governance”. The system must standardize these variations for topic modeling and clustering. Construct a morphology-based normalization mechanism that decomposes each word into its root and affixes, identifies derivational layers, and maps them to a common base form. Apply the approach to the input words and derive the normalized representation along with their morphological breakdown.

- Develop a Python program to implement a morphology-based normalization module for the input words "govern", "government", and "governance". The program should perform morphological decomposition to identify the root word and successive derivational suffixes, determine the derivational level associated with each word, normalize all variants to a common lexical representation, and generate a structured output displaying the original word, root form, detected affix(es), derivational hierarchy, normalized representation, and the final output used for topic modeling and document clustering.

4.A document classification system processes input words such as “activate”, “activation”, and “reactivation” to improve semantic grouping and indexing.
Construct a morphological parsing strategy to identify prefixes, suffixes, and root forms while distinguishing derivational layers.
Analyze how prefix addition and suffix transformation affect meaning and word class.
Apply the process to decompose each word and map them to a unified base representation.
Derive the morphological structure and normalized root for each input word.

- Develop a Python program to implement a morphological parsing and normalization system for the input words "activate", "activation", and "reactivation". The program should detect and separate prefixes, root words, and suffixes using rule-based morphological parsing, identify the sequence of derivational transformations that produce each word, analyze the effect of each morphological modification on the word class and semantic meaning, and generate a structured report showing the original word, prefix (if present), root word, suffix, derivational sequence, normalized base form, and final parsed representation for use in document classification and semantic indexing.

5. A search optimization module processes input words such as “create”, “creates”, and “creating” to ensure consistent retrieval across grammatical variations. Design a morphology-based normalization approach focusing on inflectional transformations such as tense and aspect.
Identify suffix patterns and map all variations to a common base form without altering meaning.
Apply the process to extract root forms and classify each transformation.
Determine the morphological breakdown and normalized output for each word.

- Develop a Python program to implement an inflectional morphology-based normalization module for the input words "create", "creates", and "creating". The program should recognize grammatical suffixes corresponding to different inflectional forms, identify the underlying root word by applying appropriate normalization rules, classify each input according to its grammatical feature (e.g., base form, third-person singular, present participle), and produce a structured output containing the original word, identified suffix, grammatical category, extracted root, normalized base form, and the final normalized representation to support efficient search optimization and information retrieval.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 – Search-Engine Indexing: analyzing, analysis, analytical

1. Understanding of the Scenario

A search engine receives queries in different surface forms of the same concept. If the index stores each form separately, a query for *analysis* will miss documents that only contain *analyzing* or *analytical*. The task is to decompose each word into root and affixes, decide whether each affix is inflectional or derivational, and normalise all three to one representation that becomes the index key.

2. Key Issues Identified

- **Two kinds of morphology are mixed:** *-ing* is inflectional (grammatical), whereas *-sis*, *-ic*, *-al* are derivational (new word, new class).
- **Spelling changes:** the silent *e* of *analyze* is deleted before a vowel-initial suffix (*analyz+ing*), so the stem is not identical to the root.
- **Stem allomorphs (Greek origin):** the root surfaces as *analyz-*, *analy-* (analy+sis) and *analyt-* (analyt+ic+al). A pure suffix stripper cannot connect these; an allomorph table is needed.
- **Stacked suffixes:** *analytical* has two derivational layers (*-ic* then *-al*), so stripping must be repeated right-to-left.
- **Over-stripping risk:** rules such as *-al* or *-ing* would damage words like *animal* or *king*, so every strip is validated against the lexicon.

3. Application of Concepts

Processing pipeline

1. Tokenise and lower-case the query word.
2. Try suffix rules right-to-left; accept a strip only when the remainder resolves to a known root (directly, after restoring a deleted *e*, or through the allomorph table).
3. Record each affix with its type (inflectional / derivational).
4. Classify the word: inflectional, derivational, or derivational + inflectional.
5. Normalise to the root and generate the shared index key for the posting list.

Rule table

Suffix	Type	Function	Output class	Validation guard
-ing	Inflectional	Present participle / gerund	Verb form	stem or stem+e in lexicon
-al	Derivational	Forms adjectives	Adjective	remainder is a root, or ends in hidden -ic
-ic	Derivational	Forms adjectives	Adjective	remainder in allomorph table
-sis	Derivational	Forms process nouns	Noun	remainder in allomorph table

Morphological analysis of the inputs

Word	Morpheme breakdown	Root	Type	Word-class path
analyzing	analyz(e) + -ing	analyze	Inflectional	V → V (present participle)
analysis	analy + -sis	analyze	Derivational	V → N (the process / result of analysing)
analytical	analyt + -ic + -al	analyze	Derivational (2 layers)	V → Adj (analytic) → Adj (analytical)

4. Python Program

```
"""Q1 - Rule-based morphological processing for a search-engine indexer
```

```
Words: analyzing, analysis, analytical"""

# Lexicon of known roots (guards against over-stripping) and stem allomorphs
LEXICON = {"analyze"}
ALLOMORPHS = {"analyz": "analyze", "analy": "analyze", "analyt": "analyze"}

# (suffix, type, grammatical/derivational function, resulting class)
SUFFIX_RULES = [
    ("ing", "Inflectional", "present participle / gerund (-ING)", "Verb form"),
    ("al",  "Derivational", "adjective-forming (-AL)", "Adjective"),
    ("ic",  "Derivational", "adjective-forming (-IC)", "Adjective"),
    ("sis", "Derivational", "action/process noun-forming (-SIS)", "Noun"),
]

def resolve_root(stem):
    """Map a stripped stem to a lexicon root using allomorph table."""
    if stem in LEXICON:
        return stem
    if stem + "e" in LEXICON:          # e-deletion before vowel suffix (analyz+ing)
        return stem + "e"
    return ALLOMORPHS.get(stem)

def parse(word):
    affixes, types, stem = [], [], word
    changed = True
    while changed:                     # strip suffixes right-to-left
        changed = False
        for suf, typ, func, _ in SUFFIX_RULES:
            if not stem.endswith(suf) or len(stem) - len(suf) < 3:
                continue
            cand = stem[: -len(suf)]
            # 'analytical' = analyt + ic + al : peel the hidden -ic first
            if suf == "al" and cand.endswith("ic") and resolve_root(cand[:-2]):
                affixes += ["-al", "-ic"]; types += [typ, "Derivational"]
                stem = cand[:-2]; changed = True; break
            if resolve_root(cand):     # accept only if remainder is a known root
                affixes.append("-" + suf); types.append(typ)
                stem = cand; changed = True; break
    root = resolve_root(stem) or word
    if not affixes:
        ttype = "Base form"
    elif "Derivational" in types:
        ttype = "Derivational" + (" (+ inflectional)" if "Inflectional" in types else "")
    else:
        ttype = "Inflectional"
    return {"word": word, "root": root, "affixes": list(reversed(affixes)) or ["-"],
            "type": ttype, "normalized": root}

if __name__ == "__main__":
    words = ["analyzing", "analysis", "analytical"]
    print("=" * 96)
    print(f"{'Original':<12}{'Root':<10}{'Affix(es)':<14}{'Transformation':<22}{'Normalized':<12}Index key")
    print("=" * 96)
    index = {}
    for w in words:
        r = parse(w)
        key = r["normalized"][:5]      # shared stem used as the posting-list key
        index.setdefault(key, []).append(w)
        print(f"{r['word']:<12}{r['root']:<10}{' + '.join(r['affixes']):<14}{r['type']:<22}{r['normalized']:<12}{key}")
    print("\nInverted-index posting list (unified):", index)
```

5. Program Output

```
================================================================================================
Original    Root      Affix(es)     Transformation        Normalized  Index key
================================================================================================
analyzing   analyze   -ing          Inflectional          analyze     analy
analysis    analyze   -sis          Derivational          analyze     analy
analytical  analyze   -ic + -al     Derivational          analyze     analy

Inverted-index posting list (unified): {'analy': ['analyzing', 'analysis', 'analytical']}
```

6. Decision Making and Justification

- **Decision:** normalise all three words to the lemma **analyze** and store them under the shared index key **analy**. Result: one posting list instead of three, so a query for any variant retrieves all relevant documents.

- **Why merge derivational forms here?** In a search setting the user's information need is the same whether they type *analysis* or *analyzing*; recall improves with little precision risk because the family is semantically tight.
- **Why keep the type label?** Inflection (*-ing*) is always safe to remove; derivational merging is a policy decision. Storing the label lets the ranker later down-weight derived matches or allow exact-form boosts.
- **Limitation:** the allomorph table is lexicon-driven; a production system would load it from a morphological lexicon or finite-state transducer rather than hand-code it.

Question 2 – Sentiment Platform: disagree, agreement, agreeable

1. Understanding of the Scenario

Product reviews use several words built from the same base *agree*. A sentiment model must treat them consistently: the suffixed words (*agreement*, *agreeable*) keep the positive orientation of the base, while the prefixed word (*disagree*) reverses it. The parser must therefore expose the prefix, base and suffix, and explain how each derivational change shifts the meaning.

2. Key Issues Identified

- **Prefix vs suffix effects differ:** *dis-* changes **meaning** (negation) but keeps the word class (V → V); *-ment* and *-able* change the **word class** (V → N, V → Adj) but keep the polarity.
- **Negation must not be lost:** if the normaliser reduces *disagree* to *agree* with no marker, a negative review would be scored as positive.
- **Safe segmentation:** *dis-* must be stripped only if the remainder is a valid base (*disagree* → *agree*, but *disk* → *k* is invalid).
- **Semantic drift:** *-able* adds the sense "capable of being / pleasant", so *agreeable* is a quality adjective rather than the act of agreeing.

3. Application of Concepts

Affix	Position	Type	Class change	Semantic effect
dis-	Prefix	Derivational	V → V	Negation / reversal: agree → not agree
-ment	Suffix	Derivational	V → N	Result or state of the action: the act or outcome of agreeing
-able	Suffix	Derivational	V → Adj	Capable of / worthy of: pleasant, willing to agree

Word	Prefix	Root	Suffix	Transformation	Sentiment polarity
disagree	dis-	agree	–	Prefixation (derivational)	Negative (flipped by dis-)
agreement	–	agree	-ment	Suffixation (derivational)	Positive (preserved)
agreeable	–	agree	-able	Suffixation (derivational)	Positive (preserved)

4. Python Program

```
"""Q2 - Morphological parser for sentiment analysis
Words: disagree, agreement, agreeable"""

LEXICON = {"agree"}                          # known base words
PREFIXES = {"dis": ("negation / reversal of meaning", "Verb -> Verb")}
SUFFIXES = {
    "ment": ("result / state of the action", "Verb -> Noun"),
    "able": ("capable of being / worthy of", "Verb -> Adjective"),
}
POLARITY = {"agree": "+", "disagree": "-", "agreement": "+", "agreeable": "+"}   # toy sentiment prior

def parse(word):
    prefix, suffix, base = "-", "-", word
```

```
    for p in PREFIXES:                          # rule 1: prefix stripping
        if base.startswith(p) and base[len(p):] in LEXICON:
            prefix, base = p, base[len(p):]
            break
    for s in SUFFIXES:                          # rule 2: suffix stripping
        if base.endswith(s) and base[: -len(s)] in LEXICON:
            suffix, base = s, base[: -len(s)]
            break
    cats, meaning = [], []
    if prefix != "-":
        cats.append("Derivational (prefixation)"); meaning.append(f"dis- = {PREFIXES[prefix][0]}")
    if suffix != "-":
        cats.append("Derivational (suffixation)"); meaning.append(f"-{suffix} = {SUFFIXES[suffix][0]}")
    if not cats:
        cats, meaning = ["Base form (no affix)"], ["root meaning: to hold the same view"]
    return {"word": word, "prefix": prefix + ("-" if prefix != "-" else ""),
            "root": base, "suffix": ("-" + suffix) if suffix != "-" else "-",
            "category": " + ".join(cats), "semantic": "; ".join(meaning),
            "normalized": base, "polarity": POLARITY[word]}


if __name__ == "__main__":
    for w in ["disagree", "agreement", "agreeable"]:
        r = parse(w)
        print(f"Word            : {r['word']}")
        print(f"  Prefix        : {r['prefix']}")
        print(f"  Root          : {r['root']}")
        print(f"  Suffix        : {r['suffix']}")
        print(f"  Category      : {r['category']}")
        print(f"  Semantic effect : {r['semantic']}")
        print(f"  Normalized base : {r['normalized']}")
        note = "(prefix 'dis-' flips polarity of 'agree')" if r['prefix'] != '-' else "(polarity preserved)"
        print(f"  Sentiment prior : {r['polarity']}  {note}\n")
```

5. Program Output

```
Word            : disagree
  Prefix        : dis-
  Root          : agree
  Suffix        : -
  Category      : Derivational (prefixation)
  Semantic effect : dis- = negation / reversal of meaning
  Normalized base : agree
  Sentiment prior : -  (prefix 'dis-' flips polarity of 'agree')

Word            : agreement
  Prefix        : -
  Root          : agree
  Suffix        : -ment
  Category      : Derivational (suffixation)
  Semantic effect : -ment = result / state of the action
  Normalized base : agree
  Sentiment prior : +  (polarity preserved)

Word            : agreeable
  Prefix        : -
  Root          : agree
  Suffix        : -able
  Category      : Derivational (suffixation)
  Semantic effect : -able = capable of being / worthy of
  Normalized base : agree
  Sentiment prior : +  (polarity preserved)
```

6. Decision Making and Justification

- **Decision:** all three forms are normalised to the base **agree**, but the prefix *dis-* is passed on as a **polarity flag** (equivalent to the feature NOT_agree).
- **Justification:** the base gives the sentiment model one shared vocabulary entry (less sparsity), and the flag preserves the one affix that changes sentiment. Suffix-only forms need no flag because they keep the orientation of the base.
- **Result:** *disagree* → negative, *agreement* and *agreeable* → positive, which matches how a human reader interprets the three words.

Question 3 – News Pipeline: govern, government, governance

1. Understanding of the Scenario

A news aggregator groups articles by topic. If *govern*, *government* and *governance* are counted as three separate features, the topic model spreads one theme across several words and clustering quality drops. The module must decompose each word, record the derivational layer, and map all variants to a common base.

2. Key Issues Identified

- Both noun forms are derived from the same verb root by **different suffixes** (*-ment*, *-ance*), so the stripping rules must cover both.
- The derivational **level** differs: the base verb is level 0, each derived noun is level 1. Neither noun is derived from the other, so they are siblings, not a chain.
- **Semantic drift:** *government* has become lexicalised (the institution), *governance* means the process; the normaliser accepts this loss because topic models work at theme level.
- **Sparsity:** three features for one theme dilute term frequencies, which is what normalisation fixes.

3. Application of Concepts

Word	Root	Affix	Level	Derivational hierarchy	Word-class change
govern	govern	–	0	govern (base verb)	Verb (none)
government	govern	-ment	1	govern → +ment	V → N: the body that governs
governance	govern	-ance	1	govern → +ance	V → N: the act or process of governing

4. Python Program

```
"""Q3 - Morphology-based normalization for topic modelling
Words: govern, government, governance"""

LEXICON = {"govern"}
SUFFIXES = [("ment", "Verb -> Noun", "act / result of governing"),
            ("ance", "Verb -> Noun", "state / quality of governing")]

def decompose(word):
    layers, base = [], word
    for suf, change, meaning in SUFFIXES:
        if base.endswith(suf) and base[: -len(suf)] in LEXICON:
            layers.append((suf, change, meaning)); base = base[: -len(suf)]
            break
    return base, layers

def normalize(word):
    root, layers = decompose(word)
    level = len(layers)                          # derivational depth
    hierarchy = root + "".join(f" -> +{s}" for s,   ,    in layers) if layers else root + " (level 0)"
    return {"word": word, "root": root,
            "affixes": ["-" + s for s, _, _ in layers] or ["-"],
            "level": level, "hierarchy": hierarchy, "normalized": root,
            "final": root.upper() + "_N" if level else root.upper() + "_V"}

if    name    == "    main    ":
    print(f"{'Original':<12}{'Root':<8}{'Affix':<8}{'Level':<7}{'Derivational hierarchy':<28}{'Normalized':<12}{'Topic-model token'}")
    print("-" * 95)
    tokens = []
    for w in ["govern", "government", "governance"]:
        r = normalize(w)
        tokens.append(r["normalized"])
        print(f"{r['word']:<12}{r['root']:<8}{'
'.join(r['affixes']):<8}{r['level']:<7}{r['hierarchy']:<28}{r['normalized']:<12}{r['normalized']}")
    print("\nBag-of-words before :", ["govern", "government", "governance"], "-> 3 distinct features")
    print("Bag-of-words after  :", sorted(set(tokens)), "-> 1 feature (count = %d)" % len(tokens))
```

5. Program Output

```
Original    Root    Affix   Level  Derivational hierarchy      Normalized  Topic-model token
```

```
-------------------------------------------------------------------------------------------
govern      govern  -      0     govern (level 0)          govern      govern
government  govern  -ment  1     govern -> +ment           govern      govern
governance  govern  -ance  1     govern -> +ance           govern      govern

Bag-of-words before : ['govern', 'government', 'governance'] -> 3 distinct features
Bag-of-words after  : ['govern'] -> 1 feature (count = 3)
```

6. Decision Making and Justification

- **Decision:** map all three words to the token **govern** for topic modelling and clustering.
- **Effect on features:** the bag-of-words falls from 3 distinct features to 1 feature with count 3, so term frequency for the theme is concentrated and topic coherence improves.
- **Trade-off accepted:** the distinction between the institution (*government*) and the process (*governance*) is lost; if that distinction mattered (e.g. policy analysis), the level-1 suffix label retained in the report could be used to keep them separate.

Question 4 – Document Classification: activate, activation, reactivation

1. Understanding of the Scenario

A classifier groups documents by meaning. The three words share one root but differ by prefixes and suffixes that change word class and sense. The parser must identify the layers in the order they were applied, explain the effect of each layer, and map the words to one base representation.

2. Key Issues Identified

- **Multiple derivational layers:** *reactivation* contains a prefix and three suffix layers, so the parser must peel and record them in the correct order.
- **Word-class changes:** the class changes at three steps (V → Adj → V → N) and stays the same at the prefix step (*re-*: N → N).
- **Surface fusion:** the suffixes *-ate* and *-ion* surface together as *-ation*, so a naïve single-suffix stripper would mis-segment the word.
- **Choice of normal form:** reducing everything to the deepest root *act* would merge unrelated words (*act, actor, action*). The right target is the base **activate**.

3. Application of Concepts

Derivation chain

act (V) → active (Adj, **-ive**) → activate (V, **-ate**) → activation (N, **-ion**) → reactivation (N, **re-**)

Step	Affix	Word-class change	Semantic effect
1	-ive	Verb → Adjective	Having the nature of acting (active)
2	-ate	Adjective → Verb	To make or cause to be active (activate)
3	-ion	Verb → Noun	The act or process of activating (activation)
4	re-	Noun → Noun (no change)	Adds the meaning “again”: repeating the process

Word	Prefix	Root	Suffix(es)	Layers	Normalised base
activate	–	act	-ive, -ate	2	activate
activation	–	act	-ive, -ate, -ion	3	activate
reactivation	re-	act	-ive, -ate, -ion	4	activate

4. Python Program

```
"""Q4 - Morphological parsing for document classification
Words: activate, activation, reactivation
Derivation chain:  act(V) -> active(Adj,-ive) -> activate(V,-ate) -> activation(N,-ion) -> reactivation(N,re-)"""
```

```
ROOT = "act"
# ordered derivational layers: (affix, kind, input class, output class, meaning change)
LAYERS = [
    ("-ive", "suffix", "Verb",      "Adjective", "having the nature of acting"),
    ("-ate", "suffix", "Adjective", "Verb",      "to make / cause to be active"),
    ("-ion", "suffix", "Verb",      "Noun",      "the act / process of activating"),
    ("re-",  "prefix", "Noun",      "Noun",      "again / repeated process"),
]
FORMS = {"activate": 2, "activation": 3, "reactivation": 4}   # number of layers applied

def parse(word):
    n = FORMS[word]
    applied = LAYERS[:n]
    prefix = "re-" if word.startswith("re") else "-"
    suffixes = [a for a, k, *  in applied if k == "suffix"]
    base = "activate"
    seq, cur = [ROOT], ROOT
    for aff, kind, c_in, c_out, mean in applied:
        cur = {"-ive": "active", "-ate": "activate", "-ion": "activation", "re-": "reactivation"}[aff]
        seq.append(f"{cur} [{c  out}, {aff}]")
    return {"word": word, "prefix": prefix, "root": ROOT, "suffix": " + ".join(suffixes),
            "sequence": " -> ".join(seq), "normalized": base,
            "final": f"{prefix.rstrip('-') + '+' if prefix != '-' else ''}{ROOT}+{'+'.join(s.strip('-') for s in suffixes)}",
            "effects": [(a, i, o, m) for a, k, i, o, m in applied]}

if __name__ == "__main__":
    for w in FORMS:
        r = parse(w)
        print(f"Word        : {r['word']}")
        print(f"Prefix      : {r['prefix']}")
        print(f"Root        : {r['root']}")
        print(f"Suffix(es)  : {r['suffix']}")
        print(f"Derivation  : {r['sequence']}")
        for a, i, o, m in r["effects"]:
            print(f"   {a:<5} {i:>9} -> {o:<9} | {m}")
        print(f"Normalized  : {r['normalized']}")
        print(f"Final parse : {r['final']}\n")
```

5. Program Output

```
Word        : activate
Prefix      : -
Root        : act
Suffix(es)  : -ive + -ate
Derivation  : act -> active [Adjective, -ive] -> activate [Verb, -ate]
  -ive      Verb -> Adjective | having the nature of acting
  -ate  Adjective -> Verb     | to make / cause to be active
Normalized  : activate
Final parse : act+ive+ate

Word        : activation
Prefix      : -
Root        : act
Suffix(es)  : -ive + -ate + -ion
Derivation  : act -> active [Adjective, -ive] -> activate [Verb, -ate] -> activation [Noun, -ion]
  -ive      Verb -> Adjective | having the nature of acting
  -ate  Adjective -> Verb     | to make / cause to be active
  -ion      Verb -> Noun      | the act / process of activating
Normalized  : activate
Final parse : act+ive+ate+ion

Word        : reactivation
Prefix      : re-
Root        : act
Suffix(es)  : -ive + -ate + -ion
Derivation  : act -> active [Adjective, -ive] -> activate [Verb, -ate] -> activation [Noun, -ion] -> reactivation [Noun, re-]
  -ive      Verb -> Adjective | having the nature of acting
  -ate  Adjective -> Verb     | to make / cause to be active
  -ion      Verb -> Noun      | the act / process of activating
  re-       Noun -> Noun      | again / repeated process
Normalized  : activate
Final parse : re+act+ive+ate+ion
```

6. Decision Making and Justification

- **Decision:** normalise to **activate** (the verb base), and keep the root **act** only in the parse record.

- **Justification:** the base *activate* groups all three input words and still separates them from *act*, *action* and *actor*, which belong to different semantic fields. For classification this gives better grouping than the root and less noise than the surface forms.
- **Prefix handling:** *re-* does not change the class, so it is removed for grouping but kept in the parse (re+act+ive+ate+ion) in case repetition is relevant to the document's topic.

Question 5 – Search Optimisation: create, creates, creating

1. Understanding of the Scenario

Search results must be the same whether the user types *create*, *creates* or *creating*. These forms differ only by **inflectional** suffixes, so they belong to one lexeme and can be mapped to the base form without changing the meaning.

2. Key Issues Identified

- **Inflection only:** no derivational morpheme is involved, so normalisation is meaning-preserving.
- **Silent-e deletion:** *creating* = *creat(e)* + *-ing*; removing *-ing* leaves *creat*, so the *e* must be restored by a rule checked against the lexicon.
- **Rule order for -s / -es:** *creates* is *create* + *-s*. Testing *-s* before *-es* gives the correct root immediately, whereas *-es* first would produce the wrong stem *creat*; *-es* stays as a fallback for words like *boxes*.
- **Grammatical labelling:** each form must be tagged with its feature (base, 3rd-person singular, present participle) for the report.

3. Application of Concepts

Word	Suffix	Grammatical category	Root	Rule applied
create	–	Base form (VB), present tense	create	none
creates	-s	3rd-person singular present (VBZ)	create	strip -s
creating	-ing	Present participle (VBG), progressive aspect	create	strip -ing, restore silent e

4. Python Program

```
"""Q5 - Inflectional normalization for search optimisation
Words: create, creates, creating"""

LEXICON = {"create"}
INFLECTIONS = [                                  # (suffix, grammatical category)
    ("ing", "Present participle (VBG) - progressive aspect"),
    ("s",   "3rd person singular present (VBZ)"),   # tried first: create+s
    ("es",  "3rd person singular present (VBZ)"),   # fallback: box+es
    ("ed",  "Past tense / past participle (VBD/VBN)"),
]

def analyse(word):
    if word in LEXICON:
        return {"word": word, "suffix": "-", "category": "Base form (VB) - present tense",
                "root": word, "rule": "none"}
    for suf, cat in INFLECTIONS:
        if word.endswith(suf):
            stem = word[: -len(suf)]
            if stem in LEXICON:                          # create + s
                return {"word": word, "suffix": "-" + suf, "category": cat, "root": stem,
                        "rule": f"strip -{suf}"}
            if stem + "e" in LEXICON:                    # creat(e) + ing  -> restore dropped e
                return {"word": word, "suffix": "-" + suf, "category": cat, "root": stem + "e",
                        "rule": f"strip -{suf}, restore silent e (e-deletion rule)"}
    return {"word": word, "suffix": "?", "category": "Unknown", "root": word, "rule": "no match"}

if __name__ == "__main__":
    print(f"{'Original':<10}{'Suffix':<8}{'Grammatical category':<48}{'Root':<8}{'Normalized':<11}Rule applied")
    print("-" * 115)
    for w in ["create", "creates", "creating"]:
```

```
    r = analyse(w)
    print(f"{r['word']:<10}{r['suffix']:<8}{r['category']:<48}{r['root']:<8}{r['root']:<11}{r['rule']}")
print("\nSearch index key for all three forms ->", {analyse(w)['root'] for w in ['create','creates','creating']})
```

5. Program Output

```
Original  Suffix  Grammatical category                        Root    Normalized Rule applied
-------------------------------------------------------------------------------------------------------------------
create    -       Base form (VB) - present tense             create  create     none
creates   -s      3rd person singular present (VBZ)           create  create     strip -s
creating  -ing    Present participle (VBG) - progressive aspect  create  create     strip -ing, restore silent e (e-deletion rule)

Search index key for all three forms -> {'create'}
```

6. Decision Making and Justification

- **Decision:** all three forms are indexed under **create**.
- **Justification:** inflection never alters meaning, so conflation raises recall at no precision cost. The grammatical label is retained so that tense-sensitive features can be reconstructed if needed.
- **Implementation choice:** validating candidates against the lexicon (create) and trying *-s* before *-es* avoids the wrong root *creat* that a blind suffix stripper would produce.

Summary of Results

Q	Input words	Normalised form	Main morphological point
1	analyzing, analysis, analytical	analyze	1 inflectional + 2 derivational forms; e-deletion and Greek allomorphs
2	disagree, agreement, agreeable	agree (+ NOT flag for dis-)	Prefix flips meaning; suffixes change class only
3	govern, government, governance	govern	Siblings at derivational level 1 (-ment, -ance)
4	activate, activation, reactivation	activate	4-layer chain act → active → activate → activation → reactivation
5	create, creates, creating	create	Purely inflectional; silent-e restoration